

Chapter 2

Conditional structures (in algorithmic language and C)

Problems often require the study of several situations that cannot be dealt with by simple sequences of actions. For example, to solve a second-degree equation, after calculating the determinant we need to test whether it is positive, negative or zero. The instructions `read()`, `write()` and `assign ←` cannot indicate this sign; other instructions are therefore needed : **the conditional structures.**

Conditional structures are instructions that allow you to test whether a condition true or not.

1. Definition

The conditional control structure allows a program to modify its processing according to a condition. There are three types of conditional instructions:

- Simple form
- Generalized (nested or embedded) form
- Choice form

1.1. Simple conditional control structure

It is distinguished by two forms: reduced and alternating

1.1.1. The reduced form

a. Definition :

A conditional control structure is called reduced simple form when processing depends on a **condition**. If the condition evaluates to "**true**", **processing is executed**.

Algorithm	C code
<u>If</u> (condition) Then Instruction 1 <u>End if</u>	If (condition) Instruction 1 ;
<u>If</u> (condition) Then Instruction 1 Instruction 2 Instruction n <u>End if</u>	If (condition) { Instruction 1 ; Instruction 2 ; Instruction n ; }

Example: the absolute value of a number in two versions

Positive-value algorithm1

```
Var value: integer;  
Begin  
  read(value);  
  if (value<0) then  
  {  
    value  $\leftarrow$  -1 x value;  
  }  
  write(" the positive value is: ", value);  
END
```

Positive-value algorithm2

```
Var value, positive: integer;  
Begin  
  read(value);  
  positive  $\leftarrow$  value ;  
  if (positive<0) then  
  {  
    positive  $\leftarrow$  -1 x positive;  
  }  
  write(" the positive value is: ", positive);  
END
```

1.1.2. The alternative form :

a. Definition:

A conditional control structure is considered an alternative form when processing depends on a condition **with two states**: if the condition is evaluated as "**true**", the **first process is executed**; if the condition is evaluated as "**false**", the **second process is executed**.

b. Vocabulary and syntax

Algorithm	C code
If (condition) Then Instruction 1 Else Instruction 2 End if	If (condition) Instruction 1 ; else Instruction 2 ; If (condition) { Instruction 1 ; Instruction 2 ; } else { Instruction 3 ; Instruction 4 ; } }

Example : Max-of-two-integers algorithm

Var x,y, max: integer;	// Variables have no known value at start
Begin	x=? y=? max=?
Read(x);	x= 5 y= ? max=?
read(y);	x= 5 y= 7 max= ?
if (x > y) then	The condition is evaluated: (5>7) (False)
{	This block is not executed
max ← x;	This block is not executed
}	This block is not executed
else	
{	This block is executed
max ← y;	x= 5 y= 7 max= 7
}	End of conditional block
write(" maximum: ", max);	Result display maximum : 7
END	

1.1.3. The generalized conditional control structure

a. Definition :

A conditional control structure is said to be generalised when it can solve problems involving more than two processes depending on the conditions. Execution of one process automatically causes the other processes not to be executed.

Algorithm	C code
<pre>If (condition 1) Then Traitement 1 Else If (condition 2) Then Traitement 2 Else If (condition 3) Then Traitement 3 Else Else If (condition n-1) Then Traitement n-1 Else Traitement n End if End if End if End if</pre>	<pre>If (condition) Traitement1 ; else if (condition 2) Traitement2 ; else if (condition 3) Traitement3 ; else if else if (condition n-1) Traitement n-1 ; else Traitement n ;</pre>

Example: Let's try to solve the problem of making the user read a note and displaying the comment associated with the note:

- score from 0 to 8 inclusive: "insufficient";
- score from 8 to 12 inclusive: "average";
- score from 12 to 16 inclusive: "good";
- rating from 16 to 20 inclusive: "very good";

The classic solution:

Comments-notes algorithm

```
Var  
note: integer;  
Begin  
read(note);  
  if (note  $\leq$  8) then  
    write("insufficient");  
  if (score > 8) AND (score  $\leq$  12) then  
    write("medium");  
  if (score > 12) AND (score  $\leq$  16 ) then  
    write well " " ;  
  whether (score>16) then  
    write("very good");  
END.
```

Comments-notes-best- algorithm

```
Var  
note: integer;  
Begin  
read(note);  
  if (score  $\leq$  8) then  
    write("insufficient");  
  else if (score  $\leq$  12) then  
    write("medium");  
  else if (score  $\leq$  16) then  
    write well " " ;  
  Otherwise  
    write("very good");  
END
```

Remarks :

- It is preferable to put the most possible events first.
- Each treatment can include one or more instructions.

1.1.4. The conditional choice control structure

a. Definition:

A conditional control structure is said to be choice when the processing depends on the value that a selector will take. This selector is of integer, character or boolean type.

b. Vocabulary and syntax :

Algorithm	C code
According to selector Do Value 1 : Action 1 Value 2 : Action 2-1 Action 2-2 Action 2-n Value 3, Value 4: Action 4 Value 5 .. Value 7 : Action 6 Value N : Action N Else Action R End According to	switch (selector) { case Value 1 : Action 1 ; break ; case Value 2 : Action 2-1 ; Action 2-2 ; Action 2-n ; break ; case Value 3 : case Value 4 : Action4 ; break ; case Value 5 : case Value 6 case Value 7 : Action 5 ; break ; case Value N : Action N ; break ; default : Action R ; }

2. The Condition

The condition is a simple Boolean expression or a sequence of Boolean expressions (compound condition).

A simple condition is made up of a single comparison operator (=, ≠, <, ≤, >, ≥).

A compound condition is a condition made up of several simple conditions linked by logical operators: AND, OR, exclusive OR (XOR) and NOT.

Examples:

- x between 2 and 6: (x >= 2) AND (x < =6)
- n divisible by 3 or 2: (n%3=0) OR (n%2=0)
- x is true and y is false: (x=true) and (not y)

Two values and only two are identical among a, b and c: (a=b) XOR (a=c) XOR (b=c)

Exercise 1

Algorithm exo5_1 Variable a,b,c : integer <u>Begin</u> Read(a,b) $c \leftarrow a+b$ If (c < 0) alors $c \leftarrow -(a+b)$ End If Write(c) <u>End</u>	Algorithm exo5_2 Variable a,b,c : integer <u>Begin</u> Read(a,b) $C \leftarrow a+b$ If (c < 0) alors Write(-c) End If If c ≥ 0 alors Write(c) End If <u>End</u>	Algorithm exo5_3 Variable a,b,c : integer <u>Begin</u> Read(a,b) $c \leftarrow a+b$ If (c < 0) alors Write(-c) Else Write(c) End If <u>End</u>
---	--	--

1. Execute (give the execution trace) the algorithms exo5_1 , exo5_2 and exo5_3 for :
 - a= -10 , b = 5
 - a = 10, b = -29
 - a = 5, b = -2
 - a = 8 , b = 6
 - a = -4 , b = 9
2. What do these algorithms do?
3. What is the best solution (the best algorithm)?

Exercise 2

The inhabitants of a town pay tax according to the following rules:

- men over the age of 20 pay tax
- women pay tax if they are aged between 18 and 35
- all others pay no tax

Write an algorithm that asks the age and sex ('M' or 'F') of a resident and displays whether he or she is taxable. Test the algorithm with a 20-year-old woman, an 18-year-old man and a 35-year-old man.